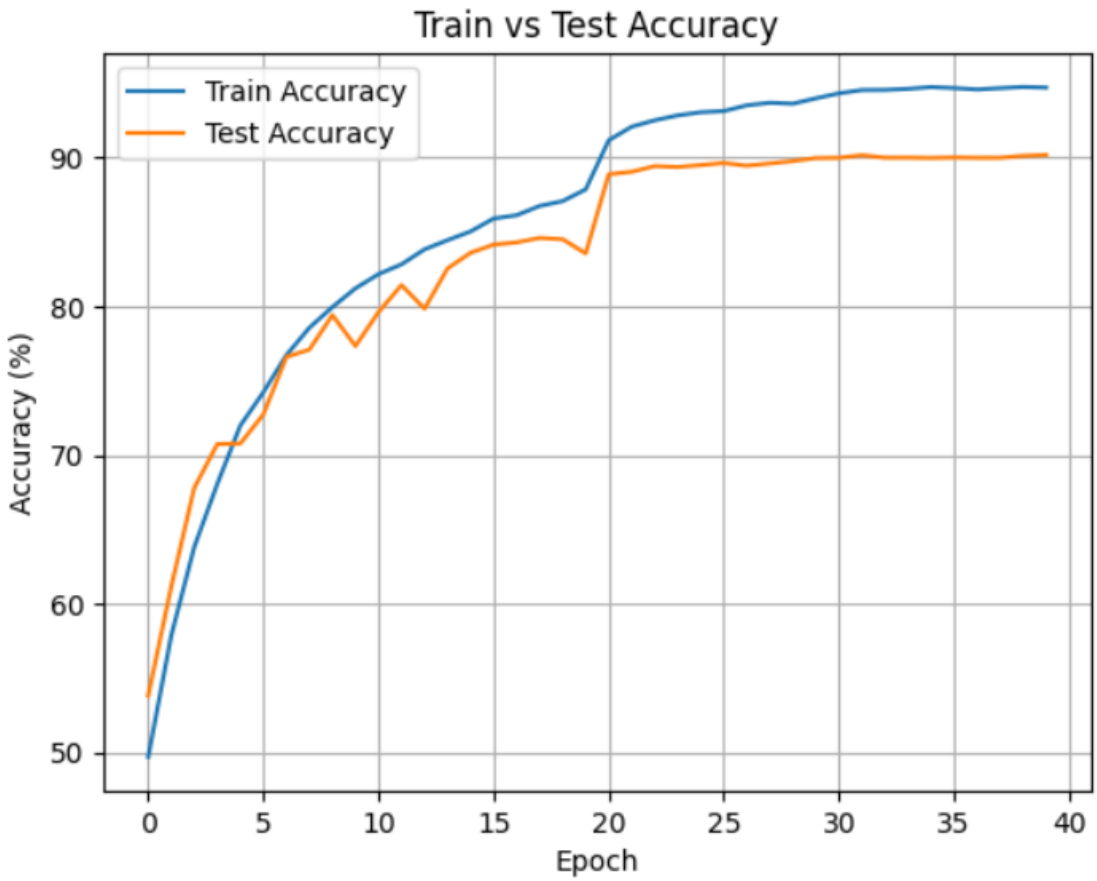
Lab02-Prunning

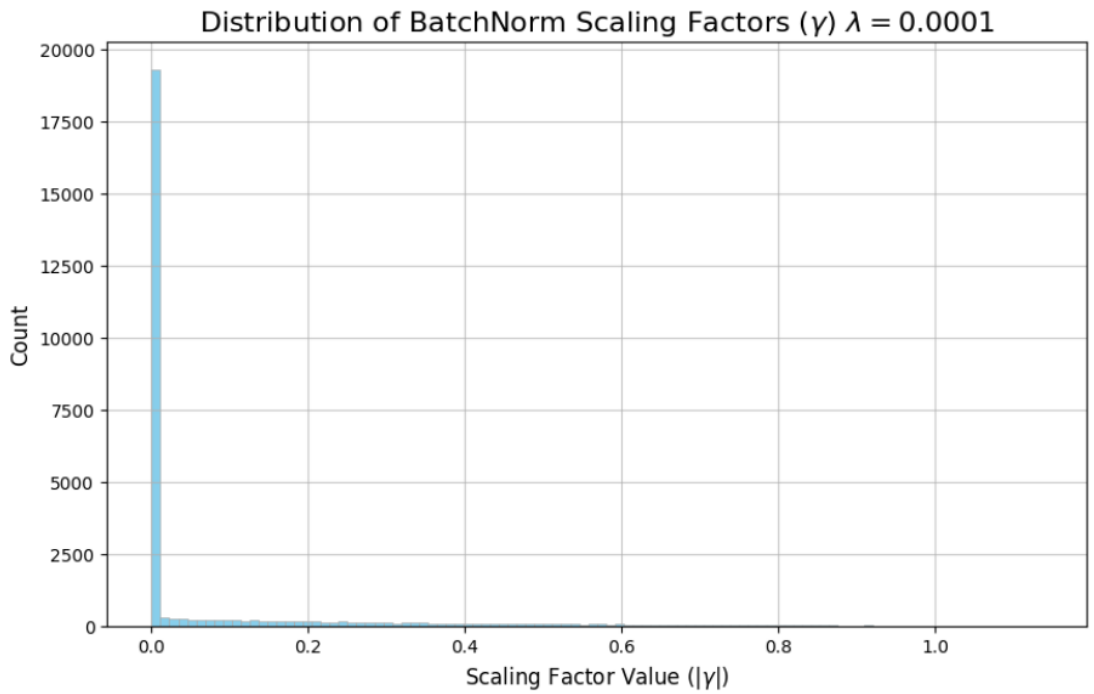
1. Plot train accuracy and test accuracy of the original model in sparsity-training over epochs (Only λ = 1e-4 are needed.)

1e-4

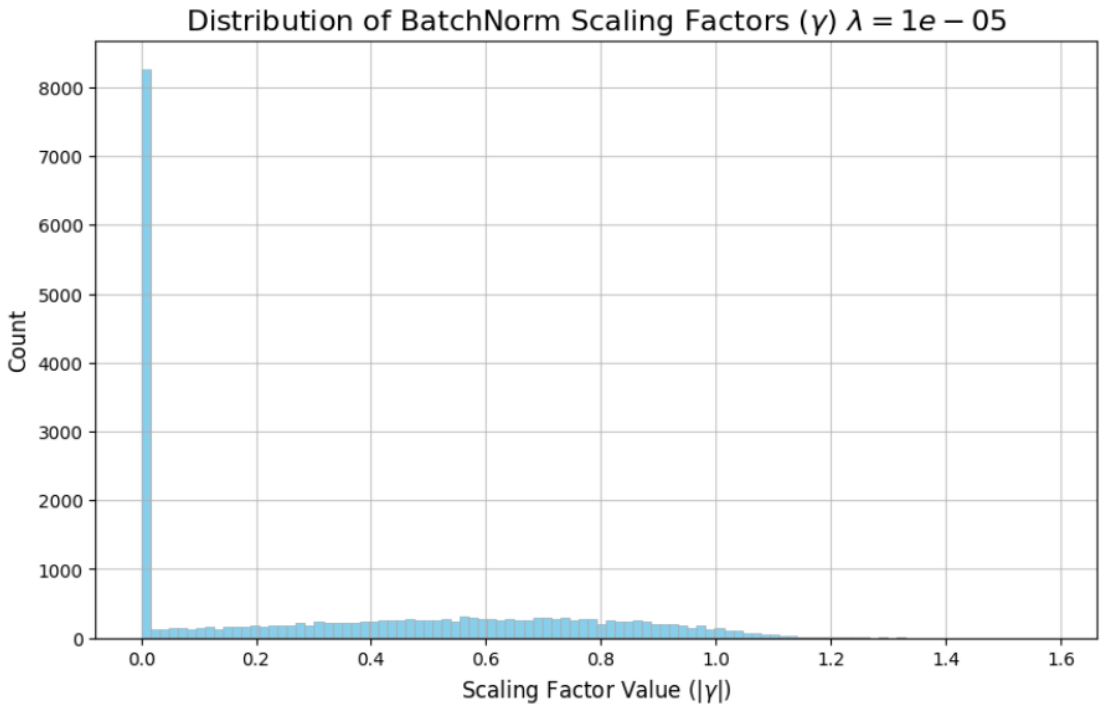


2. Plot scaling factor distribution with 3 different λ value

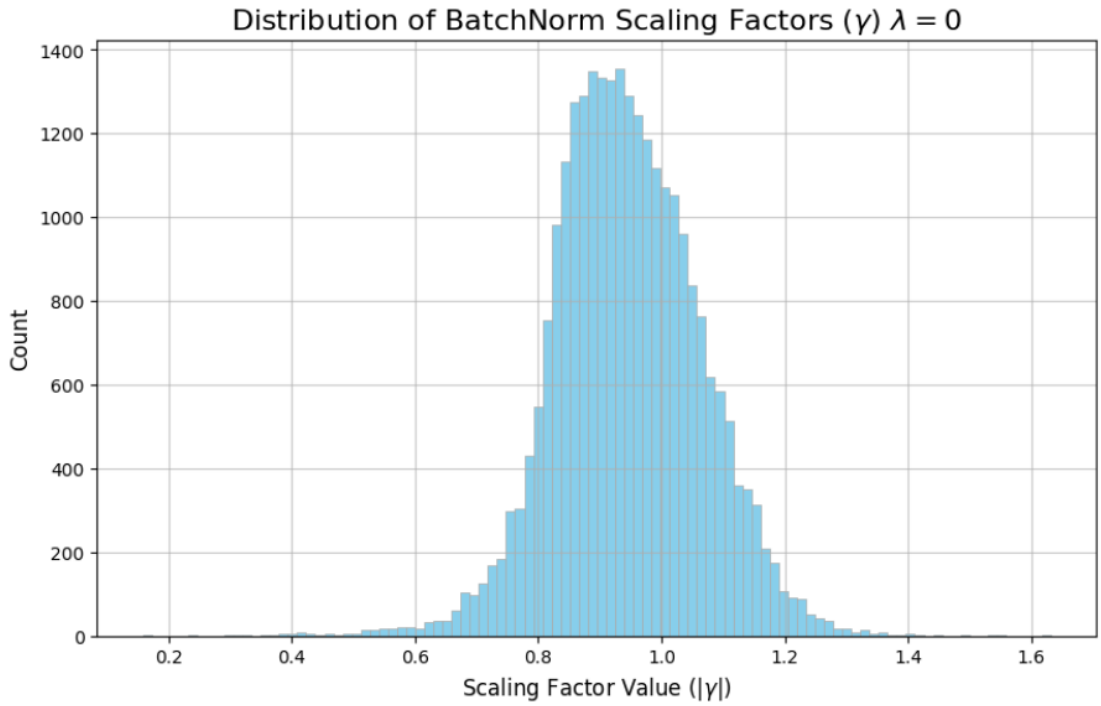
1e-4


Distribution of BatchNorm Scaling Factors (γ) $\lambda = 0.0001$
Count
Scaling Factor Value ($|\gamma|$)

1e-5


Distribution of BatchNorm Scaling Factors (γ) $\lambda = 1e-05$
Count
Scaling Factor Value ($|\gamma|$)

0



3. Show the model test accuracy after pruning 50% and 90% channels

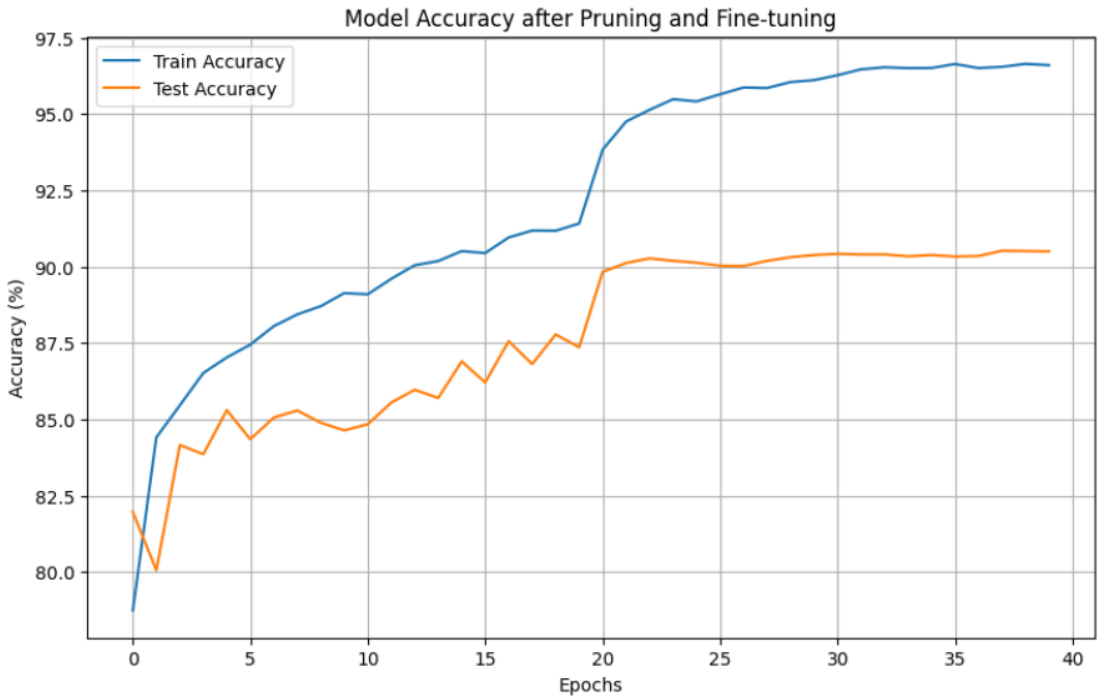
50%

```
Test set: Accuracy: 9019/10000 (90.2%)
```

90%

```
Test set: Accuracy: 1970/10000 (19.7%)
```

4. Plot train accuracy and test accuracy after fine-tuning the pruned 90% model over epochs



5. Show and compare the test accuracy and model parameters among the original and fine-tuned models

Original:

Test accuracy:

90.2%

Parameters:

23,513,162

Fine-tune:

Test accuracy:

90.5%

Parameters:

3,315,698

Parameters 從 **2000** 多萬降到 **300** 多萬，**test-acc** 的狀況我在第九

題有寫。

6. Explain how you modified resnet.py

Bottleneck:

```
self.conv1 = nn.Conv2d(in_channels, out_channels[0], kernel_size=1, stride=1, padding=0, bias=False)
self.bn1 = nn.BatchNorm2d(out_channels[0])

self.conv2 = nn.Conv2d(out_channels[0], out_channels[1], kernel_size=3, stride=stride, padding=1, bias=False)
self.bn2 = nn.BatchNorm2d(out_channels[1])

self.conv3 = nn.Conv2d(out_channels[1], out_channels[2], kernel_size=1, stride=1, padding=0, bias=False)
self.bn3 = nn.BatchNorm2d(out_channels[2])

self.relu = nn.ReLU(inplace=True)
self.downsample = downsample
```

第一個卷積層：**1x1 (kernel_size)**，將 **in_channels** 映射到 **out_channels[0]**

第二個卷積層：**3x3 (kernel_size)**，在 **out_channels[0]** 到 **out_channels[1]** 之間，執行步幅為 **stride**

第三個卷積層：**1x1 (kernel_size)**，將 **out_channels[1]** 映射到 **out_channels[2]** (最終輸出通道數)

_make_layer:

```python
def _make_layer(self, block, planes, blocks, cfg, stride=1):
    ##############################################################################
    # Figure out how to generate the correct layers and downsample based on cfg #
    ##############################################################################
    downsample = None
    out_channels = [cfg[self.current_cfg_idx + i] for i in range(3)]

    if stride != 1 or self.inplanes != out_channels[2]:
        downsample = nn.Sequential(
            nn.Conv2d(self.inplanes, out_channels[2],
                      kernel_size=1, stride=stride, bias=False),
            # nn.BatchNorm2d(out_channels[2]),
        )

    layers = []
    layers.append(block(self.inplanes, planes, out_channels, downsample, stride))
    self.inplanes = out_channels[2]
    self.current_cfg_idx += 3

    for _ in range(1, blocks):
        out_channels = [cfg[self.current_cfg_idx + i] for i in range(3)]
        layers.append(block(self.inplanes, planes, out_channels))
        self.inplanes = out_channels[2]
        self.current_cfg_idx += 3

    # 要回傳 Sequential
    return nn.Sequential(*layers)
```

從 **cfg** 中取出當前 **Bottleneck** 需要的三個輸出通道數

檢查是否需要 **downsampling**：**stride** 不為 **1** 或輸入通道數與輸出通道數不匹配

加入該層的第一個 **Bottleneck** 區塊 **(**可能帶有 **downsample** 和 **stride)**

更新下一層的 **in_channels**

cfg 索引前進 **3 (**因為用了 **3** 個通道數**)**

7. Explain how you copied the original model weights to the pruned model.

最重要的是 Conv 層，我是決定從 mask 下手，newmodel 的結構是精簡後的 cfg 決定，把判斷式寫在 mask 那邊。

不剪枝模型最開頭的 model.bn1，因此它成功地固定了第一個 Bottleneck 區塊的輸入通道(conv1 的輸入)

而 m.bn1 和 m.bn2 是可以做剪枝的，m.bn1 剪枝後輸出為 conv2 輸入，m.b2 剪枝後輸出為 conv3 的輸入，不過要固定輸出，m.bn3 就不能剪枝。

然後 m.bn1 和 m.bn2 都要注意剪枝後通道數為 0 的情況

```
# 處理 bn1 (剪枝)
weight_copy = m.bn1.weight.data.abs().clone()
if CUDA:
        mask = weight_copy.gt(threshold).float().cuda()
else:
        mask = weight_copy.gt(threshold).float()

# 注意: 需自行設計處理剩下channel數為0的情況 (e.g. 至少保留3個channel)
##########################################################################
# 如果剪枝後通道數為 0,則至少保留 1 個通道
if int(torch.sum(mask)) == 0:
        mask[torch.argmax(weight_copy)] = 1.0
##########################################################################

pruned += mask.shape[0] - torch.sum(mask)
num_remaining = int(torch.sum(mask))
cfg.append(num_remaining)
cfg_mask.append(mask.clone())
print(f"Layer (bn1) \t total channel: {mask.shape[0]} \t remaining channel: {num_remaining} (Pruned)")
```

```
# 處理 bn2 (剪枝)
weight_copy = m.bn2.weight.data.abs().clone()
if CUDA:
        mask = weight_copy.gt(threshold).float().cuda()
else:
        mask = weight_copy.gt(threshold).float()


# 注意: 需自行設計處理剩下channel數為0的情況 (e.g. 至少保留3個channel)
##########################################################################
# 如果剪枝後通道數為 0,則至少保留 1 個通道
if int(torch.sum(mask)) == 0:
        mask[torch.argmax(weight_copy)] = 1.0
##########################################################################

pruned += mask.shape[0] - torch.sum(mask)
num_remaining = int(torch.sum(mask))
cfg.append(num_remaining)
cfg_mask.append(mask.clone())
print(f"Layer (bn2) \t total channel: {mask.shape[0]} \t remaining channel: {num_remaining} (Pruned)")
```

```
# 處理 bn3 (不剪枝)
num_channels = m.bn3.weight.data.shape[0]
if CUDA:
        mask = torch.ones(num_channels).cuda()
else:
        mask = torch.ones(num_channels)
cfg.append(num_channels)
cfg_mask.append(mask.clone())
print(f"Layer (bn3) \t total channel: {num_channels} \t remaining channel: {num_channels} (Not Pruned)")
```

Weight copy 主體:

```
if isinstance(m0, nn.BatchNorm2d):
        # 將原模型中被剪掉的 channel 對應的權重清零
        m0.weight.data.mul_(end_mask)
        m0.bias.data.mul_(end_mask)

        ##################################################
        # 找出遮罩中非零元素的 index
        index = np.squeeze(np.argwhere(np.asarray(end_mask.cpu().numpy())))
        if index.ndim == 0:
                index = np.expand_dims(index, 0)
        ##################################################

        ##################################################
        # 複製 weight, bias, running mean, and running variance
        m1.weight.data = m0.weight.data[index.tolist()].clone()
        m1.bias.data = m0.bias.data[index.tolist()].clone()
        m1.running_mean = m0.running_mean[index.tolist()].clone()
        m1.running_var = m0.running_var[index.tolist()].clone()
        ##################################################

        layer_id_in_cfg += 1
        start_mask = end_mask.clone()

        # 最後一層連接層不做修改
        if layer_id_in_cfg < len(cfg_mask):
                end_mask = cfg_mask[layer_id_in_cfg]
```

```
elif isinstance(m0, nn.Conv2d):
        if isinstance(old_modules[layer_id + 1], nn.BatchNorm2d):
                idx0 = np.squeeze(np.argwhere(np.asarray(start_mask.cpu().numpy())))
                idx1 = np.squeeze(np.argwhere(np.asarray(end_mask.cpu().numpy())))
                if idx0.ndim == 0:
                        idx0 = np.expand_dims(idx0, 0)
                if idx1.ndim == 0:
                        idx1 = np.expand_dims(idx1, 0)

                ##################################################
                # 複製 weight
                w = m0.weight.data[:, idx0.tolist(), :, :].clone()    # 先刪掉不需要的輸入通道
                w = w[idx1.tolist(), :, :, :].clone()    # 再刪掉不需要的輸出通道
                m1.weight.data = w.clone()    # 將精簡後的卷積權重複製到新模型
                ##################################################

        # downsample 層不用 prune
        else:
                m1.weight.data = m0.weight.data.clone()
```

```
elif isinstance(m0, nn.Linear):
        idx0 = np.squeeze(np.argwhere(np.asarray(start_mask.cpu().numpy())))
        if idx0.ndim == 0:
                idx0 = np.expand_dims(idx0, 0)

        ##################################################
        # 複製 weight
        m1.weight.data = m0.weight.data[:, idx0.tolist()].clone()
        ##################################################

        # 複製 bias
        m1.bias.data = m0.bias.data.clone()
```

8. Please think about why it prompts you to fix the input and output channel numbers of each bottleneck to the channel numbers before pruning. What could happen if they are not fixed, and why?

ResNet 的精髓在於 **output = F(x) + downsample(x)**，這兩條路徑最後必須逐元素相加，為了讓這個 **+** 運算可以成功，**F(x)** 的維度（特別是通道數）必須與 **x**（或 **downsample(x)**）的維度完全相同。

在 **Bottleneck** 模組中，輸入通道數由 **x** 決定，**x** 會同時流向 **conv1** 和 **downsample** 層；輸出通道數由 **conv3** 的輸出 **(**即 **F(x))** 和 **downsample** 層的輸出決定。

架構：**conv1 -> bn1 -> conv2 -> bn2 -> conv3 -> bn3**

如果修剪了 **conv3**（決定輸出的層），會影響到下一個 **conv1** 的輸入和殘差路徑的最終輸出通道數；如果修剪了 **downsample** 會影

響捷徑路徑的最終輸出通道數，就會有 **mismatch** 的問題。

9. Describe the problems you encountered and how you solved them

在做 **Conv** 層的 **weight copy** 時，一直出現 **mismatch** 的問題，後來知道 **Bottleneck** 的輸入輸出需固定，所以查了一下怎麼改 **conv** 層的 **weight copy** 那邊需要有條件限制，但怎麼改都一樣有錯，後來我從 **mask** 那邊下手看看就好了。

還有我 **fine-tune** 後的 **test-acc** 比原本的還要高 **0.3%**，後來查了後發現，**0.3%**好像在正常範圍，剪枝 **+ fine-tuning** 有時會改善泛化能力，讓測試準確率超過原始模型